Learning Objectives

- Use `DeleteView` to allow users to delete a model record; override `get_success_url()` to redirect after deletion
- Distinguish `get_absolute_url()` (a model method returning a detail URL) from `get_success_url()` (a view method returning the post-action URL)
- Use `UpdateView` to render a pre-populated form and save edits to an existing record
- Retrieve model objects with `Manager.all()`, `.get()`, `.filter()`, and `.exclude()`
- Sort query results with `.order_by()` and count them with `.count()`
- Apply ORM queries inside views to pass filtered, sorted data to templates

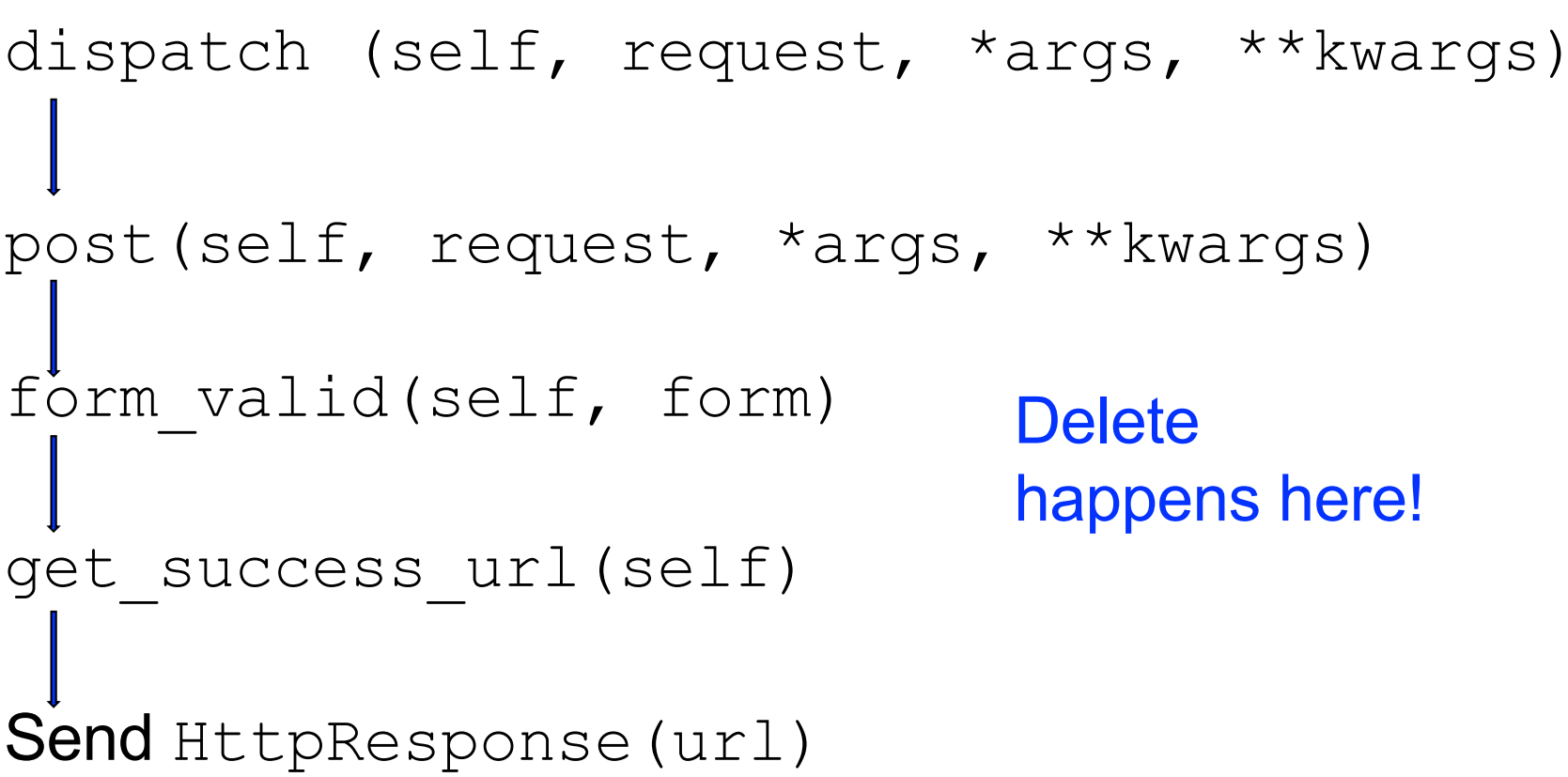
UpdateView, DeleteView

CS412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

`DeleteView`: flow chart of method calls



Generating a URL using the `reverse` function

```
class DeleteCommentView(DeleteView):

    template_name = ”blog/delete_comment_form.html"
    model = Comment

    def get_success_url(self):
        '''Return the URL to which we redirect the browser
        after successfully deleting this Comment.'''
        pk = self.kwargs.get("pk")
        comment = Comment.objects.get(pk=pk) # .get method retrieve 1 object
        article = comment.article
        return reverse('article', kwargs={'pk':article.pk})
```

And these URLs:

```
urlpatterns = [
    # ...
    path(article/<int:pk> , ArticleView.as_view(), name="article"),
]
```

get_absolute_url vs get_success_url

- get_absolute_url is a method on a model class.
  It provides a URL that will show one instance of this model. For example:

```
class Article(models.Model):
    '''Encapsulate a blog article.'''
    # data attributes:

    def get_absolute_url(self):
        '''Provide a url to show this object.'''
        return reverse('article', kwargs={'pk':self.pk})
```

- get_success_url is a method on a view class.
  It provides a URL to which the browser is directed after the view completes its task. For example:

```
class DeleteCommentView(DeleteView):

    template_name = "blog/delete_comment_form.html"
    model = Quote

    def get_success_url(self):
        '''After delete, send the browser to article page.'''
        pk = self.kwargs.get("pk")
        comment = Comment.objects.get(pk=pk)
        article = article.pk
        return reverse(article', kwargs={'pk': article.pk})
```

UpdateView: views.py

`UpdateView` works like `CreateView` but loads an existing object (via `pk` from the URL) and pre-fills the form. On POST it validates and saves the changes.

```
# blog/views.py
from django.views.generic.edit import UpdateView
from django.urls import reverse
from .models import Article
from .forms import UpdateArticleForm

class UpdateArticleView(UpdateView):
    model         = Article
    form_class    = UpdateArticleForm
    template_name = 'blog/update_article_form.html'

    def get_success_url(self):
        return reverse('article', kwargs={'pk': self.object.pk})
```

✏️ Use `fields = ['title', 'text']` on the view instead of `form_class` for a quick form without a separate `forms.py`.

`mini_insta` case study: Data Models

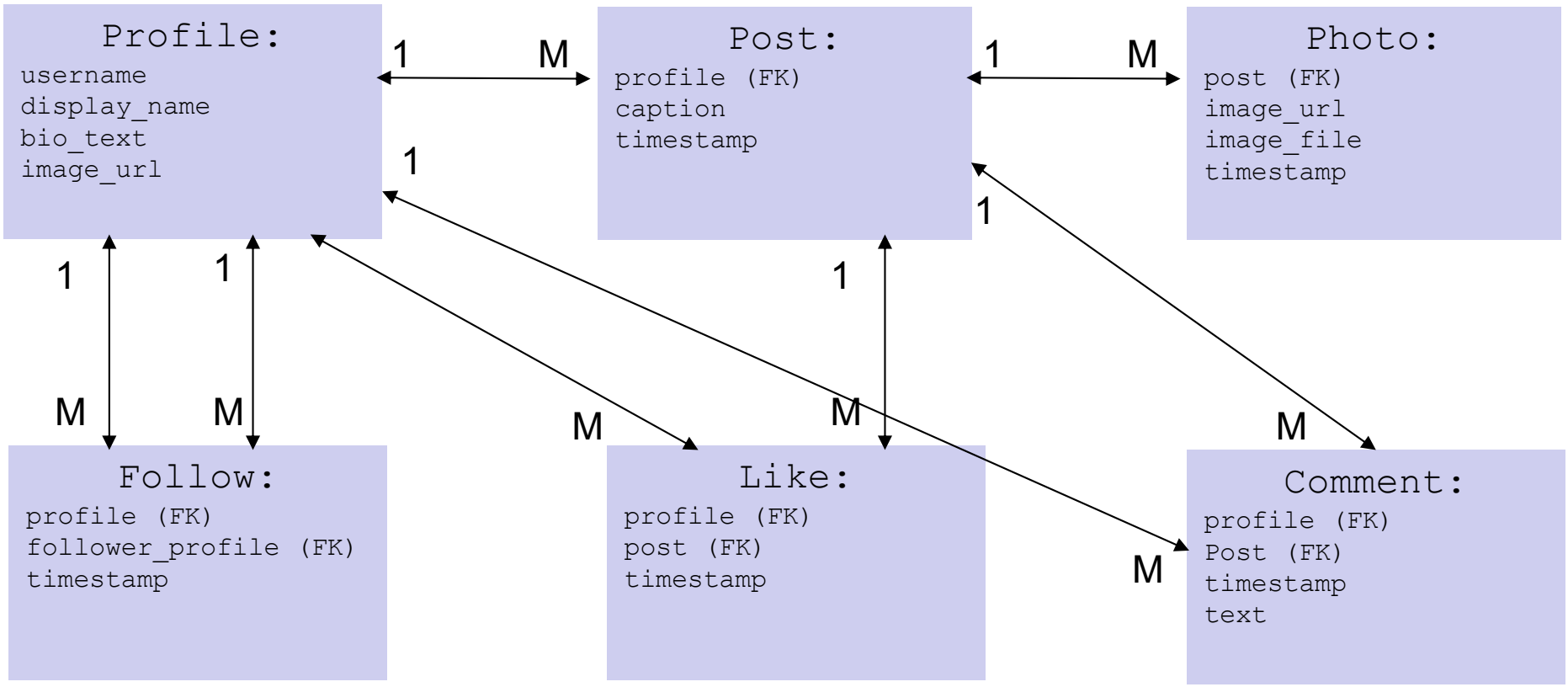
CS412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

Refining our Data Model

Previous `models.py`



Note: the `Follow` object has 2 FKs to `Profile`: the `Profile` who is being followed, and the `Profile` who is following.

Django Web Applications: Object Relational Model Queries

Computer Science 412
Boston University

Aaron Stevens

BOSTON UNIVERSITY

The django.db.models.manager.Manager

- Every Django model class has a special attribute called objects, which is an instance of `django.db.models.manager.Manager`.
- For example, in the blog application:
  - `Article.objects`
  - `Comment.objects`
- The objects class provides several methods for obtaining a `QuerySet` – the result of searching the database for objects.

Querying with the ORM

Django's ORM lets you query the database using Python — no SQL required.

```
Article.objects.all()
```

Returns all Article records as a QuerySet

```
Article.objects.filter(author="John")
```

Records where author equals "John"

```
Article.objects.get(pk=1)
```

Exactly one record; raises an error if not found

```
Article.objects.exclude(title="")
```

Records that do NOT match the condition

Manager.all()

- The objects class provides several methods for obtaining a `QuerySet` – the result of searching the database for objects.
- `Model.objects.all()` returns a `QuerySet` of all objects of a model. Example:

```
Profile.objects.all()
```

- We can limit the number of records that are returned using slicing:

```
Profile.objects.all()[:5] # 5 records
```

Manager.get()

- The Manager class provides several methods for obtaining a `QuerySet` – the result of searching the database for objects.
- `Model.objects.get(query)` returns a single object instance or an exception if not found. Example:

```
Profile.objects.get(pk=5)
```

- The query can match any data attribute of the object, for example:

```
Profile.objects.get(display_name='John')
```

Manager.filter()

- The Manager class provides several methods for obtaining a `QuerySet` – the result of searching the database for objects.
- `Model.objects.filter(query)` returns a `QuerySet` containing zero to many objects which match the query's condition. Example:

```
Profile.objects.filter(display_name='John')
```

- The query also supports near-matches, using this special syntax: (which works on any data attribute of the object)

```
Profile.objects.filter(display_name__contains='John')
Profile.objects.filter(display_name__startswith='John')
Profile.objects.filter(display_name__endswith='John')
```

Manager.exclude()

- The Manager class provides several methods for obtaining a `QuerySet` – the result of searching the database for objects.
- `Model.objects.exclude(query)` returns a `QuerySet` containing zero to many objects, which do not meet the query's condition. Example:

```
Profile.objects.exclude(name='John Lennon')
```

- The query also supports near-matches, using this special syntax: (which works on any data attribute of the object)

```
Profile.objects.exclude(display_name__contains='John')
Profile.objects.exclude(display_name__startswith='John')
Profile.objects.exclude(display_name__endswith='John')
```

ORM: Sorting and Counting

ORM QuerySets are lazy — they are not executed until evaluated. Chain methods like `.filter()`, `.exclude()`, `.order_by()`, and `.count()` to build precise queries.

```
# Sort ascending (A → Z)
Profile.objects.all().order_by("last_name")

# Sort descending (newest first)
Profile.objects.all().order_by("-timestamp")

# Count matching records
Profile.objects.filter(city="Boston").count()

# Chain filter + order_by
Article.objects.filter(published__year=2024).order_by("-published")

# Filter on a related model field (FK traversal with __)
Comment.objects.filter(article__author="Aaron")
```